

Architecture

AI-Powered Job Application Assistant

Full System Architecture Document

Stack: Python · LangGraph · LangChain · Ollama (Qwen2.5-7B-Instruct) · Streamlit · Tavily API

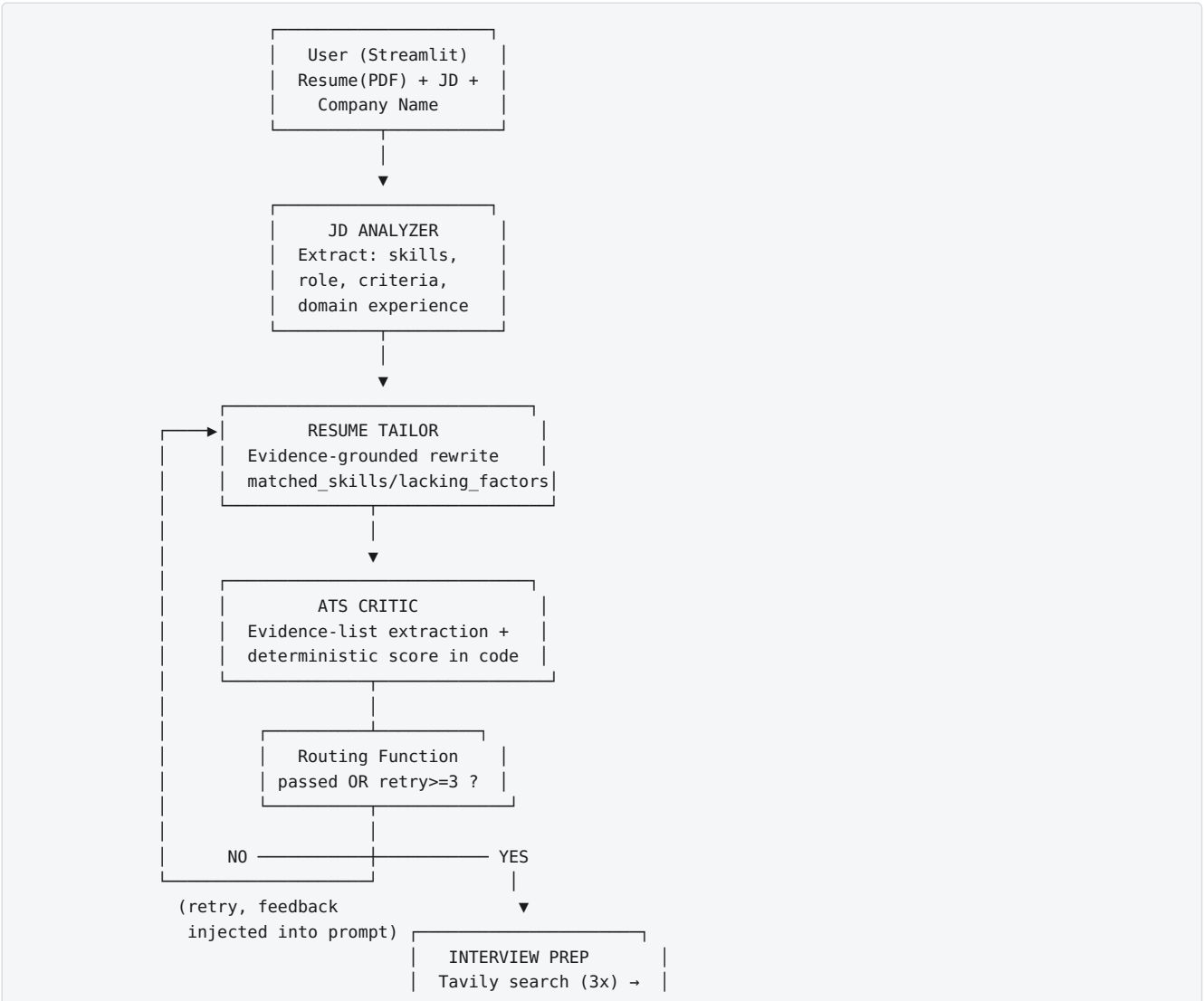
1. Project Summary

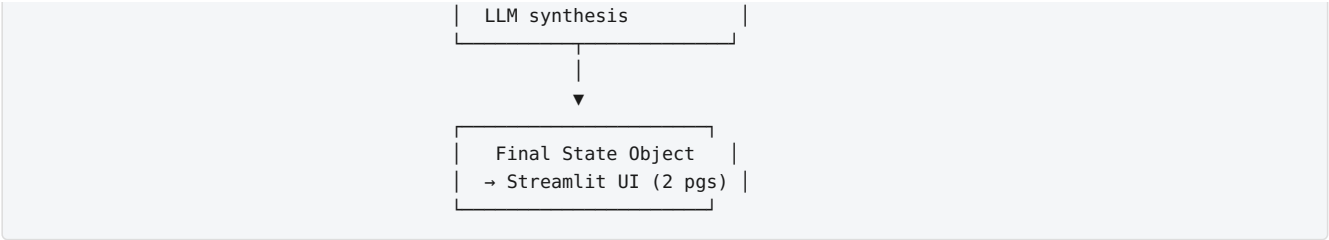
A locally-run, multi-agent AI system that takes a candidate’s resume and a target job description, and produces:

1. An ATS-optimized, truthfully tailored resume
2. A transparent ATS match score with a self-correcting retry loop
3. Company- and role-specific interview preparation, grounded in live web search

The system runs entirely on local hardware (RTX 4050, 6GB VRAM) using an open-source LLM served via Ollama, orchestrated through LangGraph as a stateful, looping agent pipeline — not a single linear prompt chain.

2. High-Level Architecture Diagram





3. Technology Stack & Why

Layer	Technology	Reason
LLM	Qwen2.5-7B-Instruct (Ollama, 4-bit quant)	Fits 6GB VRAM, strong instruction-following, free/local
Orchestration	LangGraph	Native support for stateful loops & conditional routing
Prompt/LLM interface	LangChain (langchain-ollama)	Standard, clean ChatOllama interface
Web search (grounding)	Tavily API	LLM-optimized search results, generous free tier
UI	Streamlit (multi-page)	Fast to build, native Python, session_state for cross-page data
Secrets management	python-dotenv + .env	Keeps API keys out of source control
PDF parsing	pypdf	Extracts resume text from uploaded PDF
Vector database (v2)	ChromaDB (local, persistent)	Local semantic search, no external service/API required

4. Shared State Schema

LangGraph’s `StateGraph` uses one shared `TypedDict` — every node reads the full accumulated state and returns only the fields it updates. Nothing is erased between nodes; the state only grows.

```
class PipelineState(TypedDict):
    # Raw inputs
    resume_text: str
    job_description: str
    company_name: str

    # JD Analyzer outputs
    required_skills: List[Dict]          # [{skill, importance}]
    job_role: str
    shortlist_criteria: List[str]
    domain_experience: List[str]

    # Resume Tailor outputs
    tailored_bullets: List[str]
    matched_skills: List[str]
    lacking_factors: List[str]

    # ATS Critic outputs
    score: int
    passed: bool
    missing_keywords: List[str]
    improvement_notes: str
    retry_count: int

    # Interview Prep outputs
    probable_questions: List[str]
    interview_process: List[str]
    interview_experiences: List[str]
```

5. Agent-by-Agent Breakdown

5.1 JD Analyzer

Input: `job_description` (raw text) **Output:** `required_skills` (atomic skill + importance), `job_role`, `shortlist_criteria`, `domain_experience`

Key design decisions: - Skills forced to be **atomic** (e.g. “React”, not a full sentence) via explicit instruction + worked wrong/right examples in the prompt - Each skill tagged `critical` or `nice_to_have` — this classification is later used by ATS Critic’s **deterministic Python scoring**, not left to LLM judgment - Explicit rule added to correctly interpret “X or Y” phrasing (e.g. “AWS or GCP”) as **both critical, either one satisfies it** — not as optional language - Domain/industry background (e.g. “fintech experience”) separated into its own `domain_experience` field, since it doesn’t fit the “technical skill” or “hard qualification” categories

5.2 Resume Tailor

Input: `resume_text`, `required_skills`, `job_role`, optionally `missing_keywords` + `improvement_notes` (on retry) **Output:** `tailored_bullets`, `matched_skills`, `lacking_factors`

Key design decisions: - **Evidence-grounding technique:** the model is forced to build an explicit “evidence list” of what’s literally written in the resume *before* deciding what’s matched — this measurably reduced fabrication (e.g. inferring “Docker” experience just because “AWS” was mentioned) - **Dual prompt templates:** a distinct `RETRY_PROMPT` is used when looping back from ATS Critic, explicitly incorporating prior feedback — a plain re-run of the same prompt would not “know” a previous attempt failed - **Hard rule:** never fabricate achievements, numbers, or skills not present in the original resume — a deliberate safety/ethics constraint, not just a quality one

5.3 ATS Critic

Input: `required_skills`, `tailored_bullets` **Output (LLM):** `evidence_list`, `missing_skills`, `improvement_notes` **Output (Python, deterministic):** `score`, `passed`, `missing_keywords`

Key design decisions: - **LLM does perception, code does judgment.** The LLM only extracts evidence and flags gaps; the actual score math (`critical miss = -15`, `nice_to_have miss = -5`, floor at 0) and the pass/fail threshold (`score >= 70`) are computed in plain Python — never left to the LLM’s self-reported judgment. - **Self-contradiction bug found & fixed:** early versions had the LLM’s own `improvement_notes` correctly state a skill was present, while `missing_skills` simultaneously listed the same skill as missing. Fixed by making `evidence_list` a **mandatory, visible output field** (not just an internal reasoning step), forcing the model to commit to written evidence before classifying. - **Sparse-evidence bug found & fixed:** when `tailored_bullets` was empty or entirely irrelevant (e.g. a marketing resume vs. a backend JD), the model sometimes still marked most skills as “present” — a serious false-negative. Fixed via **few-shot prompting:** a worked example showing the exact “irrelevant resume → everything missing” case was added directly into the prompt. Validated across 3 repeated runs post-fix. - **Retry loop control:** `retry_count` is incremented inside this node (not Resume Tailor), since this is the node whose output determines whether another attempt happens.

5.4 Interview Prep Agent

Input: `company_name`, `job_role` **Output:** `probable_questions`, `interview_process`, `interview_experiences`

Key design decisions: - **Search-grounded, not memory-based.** An LLM’s own training data would be stale and prone to hallucinating fake “candidate experiences” — a uniquely risky failure mode for this specific field (presenting fabricated testimony as real). Instead, 3 **targeted Tavily searches** run first (process / questions / experiences), and the LLM’s only job is to synthesize the real search results into the 3 output fields. - **Swappable search backend:** all search calls go through `agents/search_provider.py`, mirroring the same “swappable backend” pattern used for the LLM itself — Tavily is used for development (generous free tier); Google Custom Search API is the planned production swap.

6. v2 Addition: Retrieval-Augmented Style Grounding (RAG)

Purpose: the original Resume Tailor had no reference point for what strong, well-phrased resume bullets actually look like for a given role — it improvised phrasing purely from prompt instructions. v2 adds a small local knowledge base of well-written example bullets, retrieved by semantic similarity and injected into Resume Tailor’s prompt as **style inspiration only**.

This is a genuine Retrieve → Augment → Generate pipeline: - **Retrieve:** given the current `job_role` + `required_skills`, query a local ChromaDB collection of seeded example bullets using vector similarity search (not keyword matching) - **Augment:** inject the top-3 most similar examples into Resume Tailor’s prompt - **Generate:** Resume Tailor rewrites the candidate’s own bullets, informed by real phrasing patterns — but explicitly forbidden from copying any fact, number, or achievement from the retrieved examples

6.1 Components

File	Role
<code>agents/resume_examples_data.py</code>	Seed dataset — 15 hand-written example bullets across 6 roles, tagged by role
<code>agents/build_knowledge_base.py</code>	One-time script: embeds all seed examples into a persistent ChromaDB collection (<code>./chroma_db</code>) using Chroma’s default local embedding model
<code>agents/resume_retriever.py</code>	<code>get_similar_examples(query, n_results)</code> — embeds the query and returns the top-k semantically closest example bullets

6.2 A Real Bug Found & Fixed: Query Sparsity

Initial approach failed silently: querying the vector store with just the bare role name (e.g. `"Backend Engineer"`) returned results that were **not meaningfully differentiated** — similarity distances were tightly clustered (1.44–1.55 across the top 5 results), and genuine Backend Engineer-tagged examples did not appear in the top 3 at all, despite existing in the knowledge base.

Root cause: a 2-word query carries very little semantic signal compared to the full-sentence documents it’s being compared against — the embedding model had little to meaningfully differentiate on.

Fix — query enrichment: the query was rebuilt as a fuller sentence combining role *and* skills: `"{job_role} with experience in {skills}"` . This measurably widened the distance spread (1.07–1.39) and correctly surfaced genuine Backend Engineer examples in the top 3. This mirrors a well-known RAG engineering practice (query expansion) and was discovered empirically through direct testing, not assumed upfront.

6.3 Safety Guardrail: Style, Never Facts

Since the retrieved examples describe *other people’s* achievements, a real risk exists: the model could accidentally copy a fabricated statistic or accomplishment from a retrieved example into the candidate’s own resume — reintroducing a fabrication risk from a new direction. The prompt explicitly guards against this:

“These examples are from other people’s experience. Do NOT copy any specific achievement, number, or fact from these examples into the candidate’s resume. Only use them to inform tone, structure, and how technical accomplishments are typically phrased.”

Validated manually: retrieved examples containing specific fabricated metrics (e.g. *“improving query performance by 60%”*) did not leak into the candidate’s tailored output, while the *phrasing style* (e.g. *“optimizing query performance,”* *“strategic indexing”*) was clearly adopted.

7. The Self-Correcting Loop (LangGraph Conditional Edge)

```
MAX_RETRIES = 3

def decide_next_step(state: PipelineState) -> str:
    if state["passed"] or state["retry_count"] >= MAX_RETRIES:
        return "proceed"
    else:
        return "retry"

builder.add_conditional_edges(
    "ats_critic",
    decide_next_step,
```

```
        {"retry": "resume_tailor", "proceed": "interview_prep"},
    )
```

This is the core “agentic” mechanism of the system: rather than a single-shot generation, the graph automatically re-tailors the resume using the ATS Critic’s specific feedback, up to 3 times, only proceeding once the resume passes **or** the retry budget is exhausted (at which point the pipeline still completes, and the honest final score is surfaced to the user in the UI — no result is ever hidden).

Validated behavior (via manual testing across 3 resume/JD pairs):

Test case	Result
Zero skill overlap (marketing resume vs. backend JD)	Correctly failed all 3 attempts (score 45/100), consistent across all loops
Full skill overlap	Passed on attempt 1 (score 100/100)
Partial skill overlap	Correctly identified exact gaps, passed on attempt 1 (score 75/100)

8. Streamlit UI Architecture

```
job-application-agent/
├─ app.py                # Page 1: inputs, pipeline trigger, resume/ATS results
├─ pages/
│   └─ 1_Interview_Prep.py # Page 2: 3-tab interview prep view
├─ graph.py              # LangGraph StateGraph definition + node wrappers
├─ agents/
│   ├── jd_analyzer.py
│   ├── resume_tailor.py
│   ├── ats_critic.py
│   ├── interview_prep.py
│   ├── search_provider.py # Swappable Tavily/Google search wrapper
│   ├── resume_examples_data.py # RAG seed dataset (15 example bullets)
│   ├── build_knowledge_base.py # One-time ChromaDB embedding script
│   └─ resume_retriever.py    # RAG similarity search function
├─ chroma_db/             # Persistent vector store (committed - small, self-contained)
├─ .env                   # TAVILY_API_KEY (never committed)
└─ .gitignore              # venv/, .env, __pycache__/
```

- **Page 1 (app.py):** collects resume (PDF), job description, company name → extracts PDF text via `pypdf` → validates inputs → invokes the compiled LangGraph app → stores the full final state in `st.session_state["final_state"]` → renders tailored bullets + ATS score breakdown
- **Page 2 (Interview Prep):** reads from `st.session_state`, guards against direct navigation before generation, renders 3 tabs (`st.tabs`) for questions / process / experiences
- **Why `st.session_state`:** Streamlit re-runs the entire script on every interaction; without session state, results would be lost the moment the user navigated to Page 2 or interacted with any other widget

9. Known Limitations (documented, not hidden)

1. **Local 7B model inconsistency:** even with evidence-grounding and few-shot examples, occasional run-to-run variance remains in skill-detection recall — a known, inherent property of smaller open-source models, not fully eliminated by prompting alone.
2. **VRAM ceiling:** a larger model (e.g. Qwen2.5-14B) would likely improve consistency further, but exceeds the 6GB VRAM budget and requires CPU offloading, significantly increasing latency across the 4-agent pipeline — a deliberate speed/accuracy tradeoff given hardware constraints.
3. **Ambiguous “preferably X or Y” phrasing:** JD Analyzer’s critical/nice_to_have classification can be genuinely ambiguous on softer alternative-phrasing (e.g. “preferably AWS or GCP”) — a defensible gray area rather than a clear bug.
4. **Small RAG seed set:** the style-grounding knowledge base currently contains 15 hand-written examples across 6 roles — enough to demonstrate the retrieval mechanism working correctly, but a production version would benefit from a larger, more diverse example set for less common roles.

5. **No production deployment yet:** the system currently requires local Ollama + model download; a live public deployment would require either self-hosted GPU infrastructure or swapping to a cloud LLM API via the existing `model_router.py` abstraction.
-

10. Key Engineering Principles Applied Throughout

- **LLM does perception; code does judgment.** Every irreversible or high-stakes decision (pass/fail, score math, retry gating) is computed deterministically in Python — never trusted to LLM self-report.
 - **Evidence-first reasoning.** Forcing models to externalize intermediate evidence (as a real, visible output field) measurably reduces both fabrication and self-contradiction, compared to asking for a judgment directly.
 - **Swappable backends.** Both the LLM (`model_router.py`) and the search provider (`search_provider.py`) are abstracted behind thin wrapper functions, so models/providers can be swapped per-agent without touching agent logic.
 - **Grounding over memorization.** Any content with real-world factual risk (interview experiences, company-specific info) is sourced from live search rather than LLM training memory, to avoid confidently-stated hallucination.
 - **Test across the full outcome spectrum.** The system was validated not just on “does it work,” but explicitly against zero-match, full-match, and partial-match scenarios before being considered complete.
-

Document generated as a project reference — reflects the system as of the initial working build.